University of Essex
School of Computer Science and Electronic Engineering

CE264 Digital Systems Design – SPRING Term 2025
Northwest University of China

Laboratory Stage 1 – Guided design of clock divider and 7-segment display driver

This is a guided design to teach you how to work on hierarchical designs. You must have completed the tutorial, which showed you how to enter a flat circuit design and then build, download and test the design on the Basys3 board. The process for building a hierarchical design is almost the same, so we won't repeat material from the tutorial unless we really need to. Unlike the tutorial, this one does not give detailed step-by-step instructions on Multisim[1] – you must read the Multisim Help information to find the details.

There are two designs to be completed in this document: a clock divider, and a 7-segment display driver circuit. Each of these should be designed to be used as a hierarchical block: a circuit that implements a specific functionality and you can reuse in higher-level circuits of subsequent laboratory stages as if it were a PLD component in the component database (e.g., a flip-flop, a counter, etc.), without having to re-implement it. Practically, this means you need to save each of these circuits in their own Multisim design file (*.ms14) within the same working project of your overall assignment. This will allow introducing the hierarchical block into the higher-level circuit using Ctrl+H shortcut or the "Hierarchical Block from file…" button of Multisim's toolbar, as shown in the figure below.

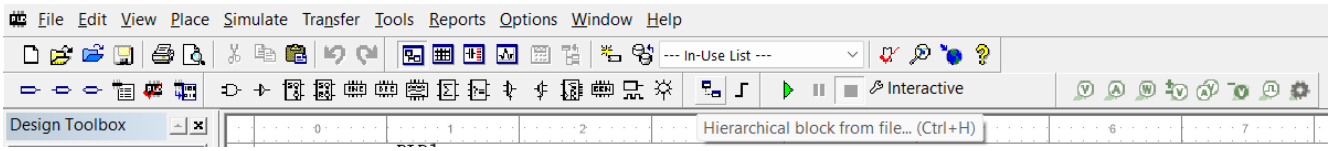

Figure 1. Inserting pre-saved hierarchical blocks into higher-level circuit designs

You are expected to complete at least the clock divider (with all three outputs working correctly) before the end of Week 1 , that is, after a total of 10 hours of laboratory time including the tutorial. Most students should not need 8 hours of laboratory time to complete this task, so your aim should be to achieve more than this and be substantially started on the 7-segment driver circuit by that time.

Hierarchy

A vital aspect of any non-trivial digital design is the correct use of hierarchy. This means that your design is broken down into sub-systems, not designed as a single (or flat) circuit. The sub-systems are represented on schematics by blocks or components, and 'inside' each one there is a schematic showing how the block is implemented. Another way to think of this is as nested schematics – schematics contain blocks, and blocks contain (other, simpler) schematics, and so on.

[1] Multisim Help is available when running Multisim (F1 key), but it has also been made available on Moodle (it is in a Windows compiled HTML (CHM) file which you can read on any Windows computer, or on Mac OS X using a CHM reader – there are free ones available on the Mac AppStore – one option is CHMReader).

There are several important advantages of using hierarchy:

1. It makes the design easier to understand and to work on. This is because you don't see the whole design at once in a huge, cluttered schematic, which is confusing. Once you understand the function of a block (how it responds to inputs and what outputs it produces), you don't need to think about or see the detailed circuit implementation of it in order to understand how the block functions as part of a larger schematic. In software, the same goal is achieved by the use of functions/procedures. for example, a function called "sqrt" may compute a square root: to use it you write sqrt(x), and you don't need to see the function's code every time you use it.
2. It makes re-use possible. Once you have designed a block and tested it, you can use it any number of times in the same design, or indeed in different designs. In software, this is usually referred to as a library (of subprograms, functions, packages, modules). Using the block does not mean copying and pasting the schematic, it means inserting the block from a library, just as you do with components like counters from the Multisim PLD library. What is inserted in a schematic is a reference to the block. Any future change to the schematic inside the block is automatically included in any circuit that uses the block (when that circuit is rebuilt or reimplemented).
3. Once a block is completed and tested, there is no need to edit it again, and it is possible from the timestamps on the files to be sure that the block has not been altered since it was last tested and approved.

Test Circuits

It is important to test blocks or sub-systems on their own if possible, before they are integrated with other sub-systems. Testing the complete system without previous sub-system testing is likely to result in failure – your system won't work when you download it to the Basys3 board, and you will have no idea why. The same is true if you only test a hierarchical block in simulation. **Multisim simulation is not an acceptable form of testing in this project, and you should not use it at all!** It does not guarantee that a circuit that seems to be working in simulation, it will also work on the Basys3 board correctly. Simulation in Vivado provides better guarantees, however, it is still not recommended: the purpose of this project is to learn to design digital circuits and execute your design in hardware, on an actual FPGA. Therefore, the recommended testing solution is for each block that you design to create a test circuit: a top-level schematic containing a copy of the block (the circuit-under-test or CUT), suitably connected to some inputs and outputs to enable you to apply input signals and observe output signals. The test circuit can be simulated or it can be designed to be tested on the Basys3 board, using input signals, switches, LEDs, output pins connected to an oscilloscope etc.

Multisim and Hierarchy

Understanding hierarchy is important, but in order to use the idea, you need to know the specific details of how Multisim handles it (this differs from one CAD system to another, but not in very significant ways).

Read the Multisim Help – look in the Contents for Schematic Capture → Working with Larger Designs → Hierarchical Design, then Adding a Hierarchical Block. There are multiple ways of creating blocks (make the block first then the schematic inside it, or **make the schematic first, then convert it into a block—this one is recommended as I tonly involves saving the circuit in its own *.ms14 file in the same project folder**, or select a group of

components and make them into a block). Use whichever method you prefer, but once you have done this a few times, you may find that one way is better than others.

An important aspect of hierarchy is naming of inputs and outputs on a block. Use generic names such as CLOCK, A, B, Y, Z, ENABLE, SELECT, not names specific to how the block might be used in the particular design you are working on. Don't confuse the names inside a block with the names of signals on a schematic that the block is used on. (This is very similar to the naming of parameters in software – you should not mix up actual parameters with formal parameters.)

The top-level of your design should contain the inputs/outputs of the FPGA. The top-level schematic may be a test circuit that you make to test a block, or it may be a complete design, ready for testing.

When you export a design or a test circuit to VHDL for testing on the Basys3 board make sure you know which VHDL file is the top-level file (Vivado needs to know this and if you get it wrong, there will be errors and your design will not build).

Clock Divider

At this point you should create a new project, as you did for the tutorial. **Use this project for the rest of the term, and put all your work inside it, so that you build up more and more blocks and top-level circuits.**

The first component you should create and test is a clock divider. This is a simple design based on counters. The Basys3 board has a 100MHz crystal clock. Later in the term you will need a 25MHz clock for generating video timing signals, and for the purposes of Stage 1 you will also need a slower clock with a frequency in the kHz range to multiplex your seven-segment display (see later).

It is also useful to have a clock with a frequency of about ½Hz or 1Hz to drive an LED (this will allow you to see when a circuit is operating by driving an LED with the ½ or 1Hz clock).

To create a clock divider, make a new block, with the following inputs and outputs:

1. Clock input: this will come from the 100MHz master clock when the clock divider is used on a schematic (call this just CLOCK). Don't connect it directly to the clock signal on the board. Do that in the external circuit where this clock divider is used.
2. 25MHz output (call this CLOCK_A).
3. 500Hz output (call this CLOCK_B). 4. ½Hz output (call this CLOCK_C).

To produce a 25MHz output, you need to divide the 100MHz input by four, using a two-bit binary counter. The easiest way to do this is to use a 4-bit binary counter from the library, and take the 25MHz output from the third most significant bit. The CNTR_4BIN_S is a suitable counter. Work out how to connect the various inputs by reading the data sheet, and if in doubt, ask.

Your clock divider is a clear sub-system that should be packaged as a single hierarchical block and schematic, and tested independently of other sub-systems. Therefore you will need a test circuit that will connect the clock input of the divider to the 100MHz clock on the Basys3 board, and the outputs to suitable output pins on the board where you can connect an

oscilloscope (to see the waveform and measure its timing). The 1Hz output is easily tested by connecting it to an LED. When you have chosen the output pins, you will need to edit a constraints file (this was described in the first tutorial, but there you were given a file to use – now you need to edit one).

Once you have entered a design that can produce the 25MHz output, test it. Do not spend a lot of time creating a fully detailed design before you attempt to test the design on the Basys3 board. It is much better to work incrementally, and add more circuitry to an already-working design. In this way, you will find out early in the process whether your design approach is correct, and if not you can change your design. To avoid errors with unconnected outputs, you can connect your 25MHz output to all three outputs on the test circuit. Implement the test circuit (containing your clock divider) and download it to the board for testing. Modify the design and re-test until the correct outputs are obtained. Check the timing of the 25MHz output with an oscilloscope to make sure that it has the correct frequency (measure it). For oscilloscope testing, the idea is to connect the corresponding clock divider output to one of the Basys3 JBx pins, and then probe this pin (and the GND) with the oscilloscope's probes. Read about the possibilities in the Basys3 manual that can be found on Moodle. A suitable template constraints file is also provided.

To produce the other two outputs, you will need decimal counters, connected in a chain. The CNTR_BCD_S will do what you need. The first counter in the chain should be clocked with the 100MHz clock. The ripple carry output of this counter will be true once in every 10 cycles of the 100MHz clock. Therefore the frequency of the ripple output will be 10MHz. Use the ripple carry output to enable a second counter so that it produces 1MHz, then six more can produce 1Hz. Because the ripple output is a narrow pulse, you will need to use a toggle flip-flop (FF_T_CLR_CO) to divide it by two and obtain a square-wave (i.e., an output "clock" signal with 50% duty cycle). This is why the frequencies have been chosen at 500Hz and ½Hz, rather than 1kHz and 1Hz. (The square-wave is needed to switch an LED on and off so that the on-time is long enough for you to see it.) As with the 25MHz output, measure the frequency on the oscilloscope to be sure that everything is working correctly.

7-segment display driver multiplexer

The Basys3 board contains a four-digit seven-segment display capable of displaying numeric values from 0000 to 9999. This display is driven by pins on the FPGA, but there is no circuit provided to drive the display – this must be done inside the FPGA, and your next task is to design, implement and test such a circuit. It is easily tested because you can see the output on the display when it is working correctly. You will need your clock divider block from the previous section to produce a suitable kHz range clock for multiplexing the display.

Read the information about the 7-segment displays in the Basys3 Reference Manual section 8.1 before you start. If you don't understand how multiplexing works, read about it in the Basys3 Reference Manual. There is also a fairly clear account of the concept of multiplexing in general on Wikipedia. If you can't recall the difference between a cathode and an anode, revise it, or look it up.

Your driver circuit should have four 4-bit inputs for data (4 BCD digits), and a clock input for the multiplexer. There should be outputs for connection to the 7-segment displays (four anode lines and seven cathode lines). Within your design, you'll need a decoder from 4-bit BCD to the seven segments, which is easy because such a decoder is provided by Multisim (it is

called DEC_BCD_7). Note that because of multiplexing, you need only one decoder (think about this – why don't you need four of them?).

Before designing the circuit inside the 7-segment driver, it is helpful to design the test circuit, so that you are clear what the inputs and outputs are. The test circuit must supply a clock with a frequency in the kHz range. That's easy – use the clock divider that you have already designed, and use the 500Hz clock as the clock to the 7-segment driver. The data inputs (four 4-bit BCD values) can come from the switches on the Basys3 board – there are conveniently sixteen of these. The outputs are easy: they need to be connected to the cathodes and anodes of the 7-segment display.

Once you have the test circuit and the block itself entered, you will need to design the multiplexing circuit itself. The main idea here is that one digit is driven at a time, cycling around the four digits so fast that the human eye cannot see any flicker. As the circuit drives each digit, you need to apply the correct data to the pins of the display, to light up the correct segments and enable one digit out of the four. Key components to do this are multiplexers and a demultiplexer and a counter to produce a cyclically repeating 2-bit code (two bits give four combinations). You will need a 4-bit multiplexer with four sets of data inputs. This does not exist in the PLD library, so it is another block that you will have to create. You will however find the MUX4_1 useful as a component within your multiplexer. If you have understood how to employ hierarchy, creating a multiplexer will not be difficult, but if you have not properly understood the idea, you will probably create a much more complicated circuit than you need to. If in doubt ask.

As with the clock divider, test the circuit incrementally. Do not wait until you have designed the entire circuit before you attempt to download to the board. Experienced designers can do this, but as a student it is better to learn early on whether your partial design works, and if not, to correct it. So, think about what tests you can do on a partial design. Can you drive one digit of the display, without multiplexing, for example? (This would enable you to check whether your 7-segment decoder is wired correctly.)

Dr Serafeim Perdikis
CE264 NWU Module Supervisor

Acknowledgements: This document is based on material provided by former University of Essex Senior Lecturers Dr Steve Sangwine and Dr Liang Hu.